

Securing cloud infrastructure

The foundational practices that close your half of the door.

Dr. Feras Shahbi

SEC 4342 · Cloud and Virtualization Security · King Khalid University



From "who secures what" to "how you secure it"

The provider secures the cloud, you secure what is in it, and the top breaches are all on your side. This chapter is the toolkit for your side.

LAST CHAPTER

The customer half is where it breaks

- Misconfiguration is the number one cloud threat
- Identity and APIs come next
- All of it is yours to control
- None of it is the provider being hacked

THIS CHAPTER

Six practices that secure your half

- Guardrails on every account
- Hardened, benchmarked baselines
- Full visibility and logging
- Posture management that catches misconfig
- Vulnerability management and remediation
- Automation that enforces it all



True, or misconception?

Four claims about running a secure cloud. Vote now; we settle them at the end.

S1

Once I have set my cloud up securely, it stays secure.

S2

Turning on all that logging is costly overkill; I will enable it if there is an incident.

S3

Patching in the cloud is like on my laptop: log in to each server and run the update.

S4

A secure score of 100 percent means my cloud cannot be hacked.



Four principles under every practice

Before the tools, the mindset. These principles, drawn from the AWS Well-Architected security pillar and the Azure security baseline, decide every choice that follows.



Secure by default

Start locked down and open only what is needed. New resources should be safe the moment they exist.



Defense in depth

Never rely on one control. Layer identity, network, host, and data so one failure is not fatal.



Least privilege

Every user, service, and machine gets the minimum access to do its job, and no more.



Assume breach

Design as if an attacker is already inside: log everything, limit blast radius, and be ready to respond.

Principles from the AWS Well-Architected Framework security pillar and Microsoft Azure security baseline.



Six foundations, one secure base

The rest of the chapter builds these six, in order. Together they are the minimum secure foundation for any cloud estate.



1 Guardrails

Org-wide rules that stop bad things by default.



2 Hardened baselines

Benchmarked, patched, secure starting images.



3 Visibility

Turn on all the logs. See everything.



4 Posture management

Continuously check config against benchmarks.



5 Vulnerability management

Find, prioritise, and remediate weaknesses.



6 Automation

Enforce and fix it all as code, continuously.



PART A · FOUNDATIONS 1 AND 2

Build on solid ground: guardrails and baselines

Stop whole classes of mistake before anyone can make them.



Guardrails: rules the whole estate obeys

Guardrails are org-wide rules enforced above individual accounts, so a mistake in one account cannot break policy. They set the outer limits; teams move freely inside them.



AWS

- Organizations groups all accounts
- Service Control Policies (SCPs) set hard limits
- Example: deny any region outside approved ones
- Control Tower builds a secure landing zone



Azure

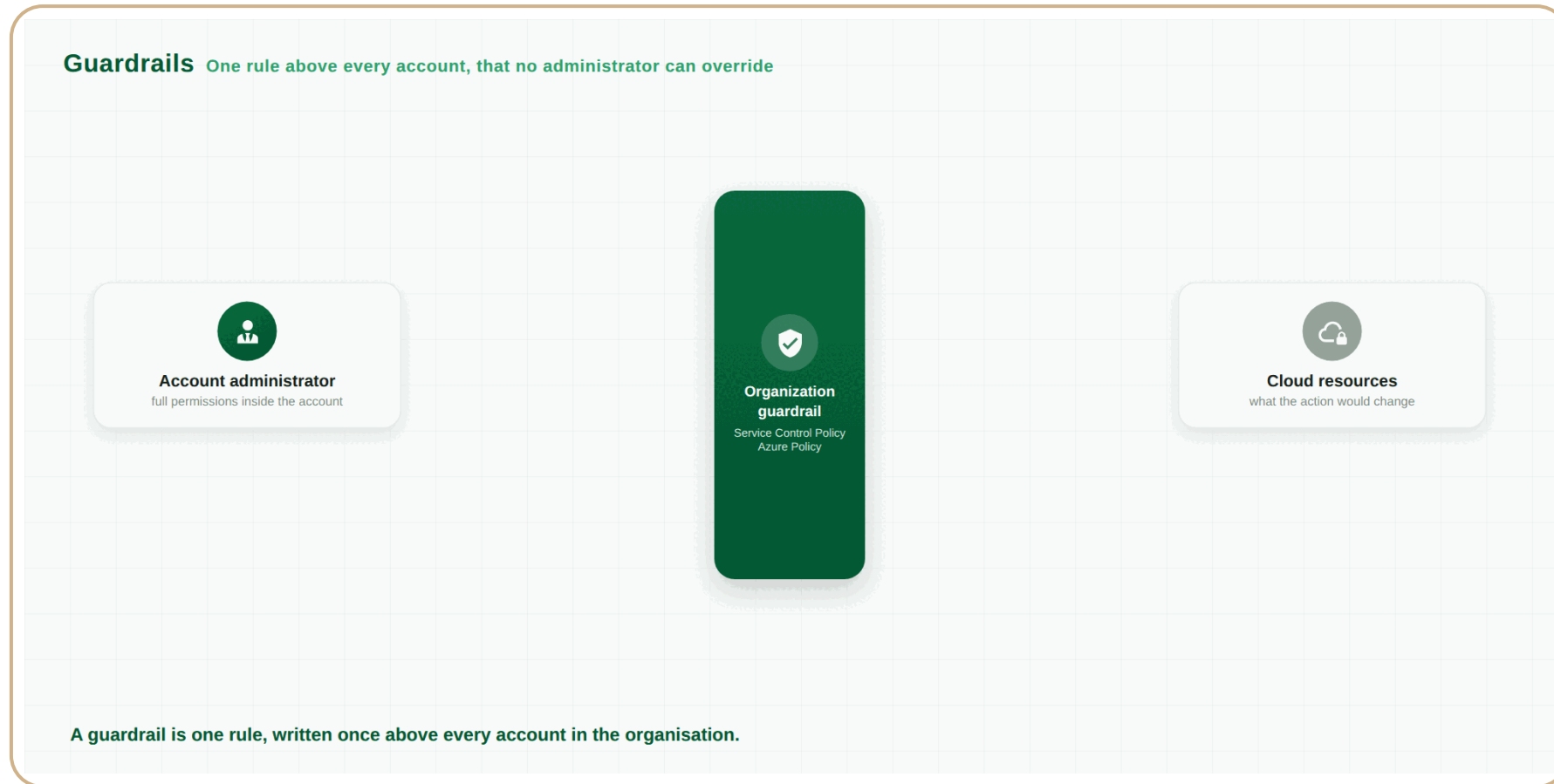
- Management Groups group all subscriptions
- Azure Policy allows, denies, or audits config
- Example: deny storage without encryption
- Landing zones ship a secure baseline

A guardrail is not a blocker: it prevents the dangerous, not the useful. Teams still self-serve, just inside safe limits.

AWS Organizations and Service Control Policies; Microsoft Azure Management Groups and Azure Policy.



A rule that no administrator can override



Ordinary work passes straight through. The action that would make a bucket public is refused above the account, where no permission inside it can reach.

Hardened baselines: start secure

Every server, image, and service should start from a known-good, hardened configuration, not vendor defaults, which favour convenience over security.



Benchmarks

The CIS Benchmarks give a consensus secure configuration for AWS, Azure, operating systems, and containers. Measure against them.



Golden images

Build one hardened, patched image and launch every server from it, so nothing starts from an unknown state.



Patch by replacing

Do not hand-patch servers. Rebuild the golden image and redeploy, so drift and forgotten servers cannot linger.



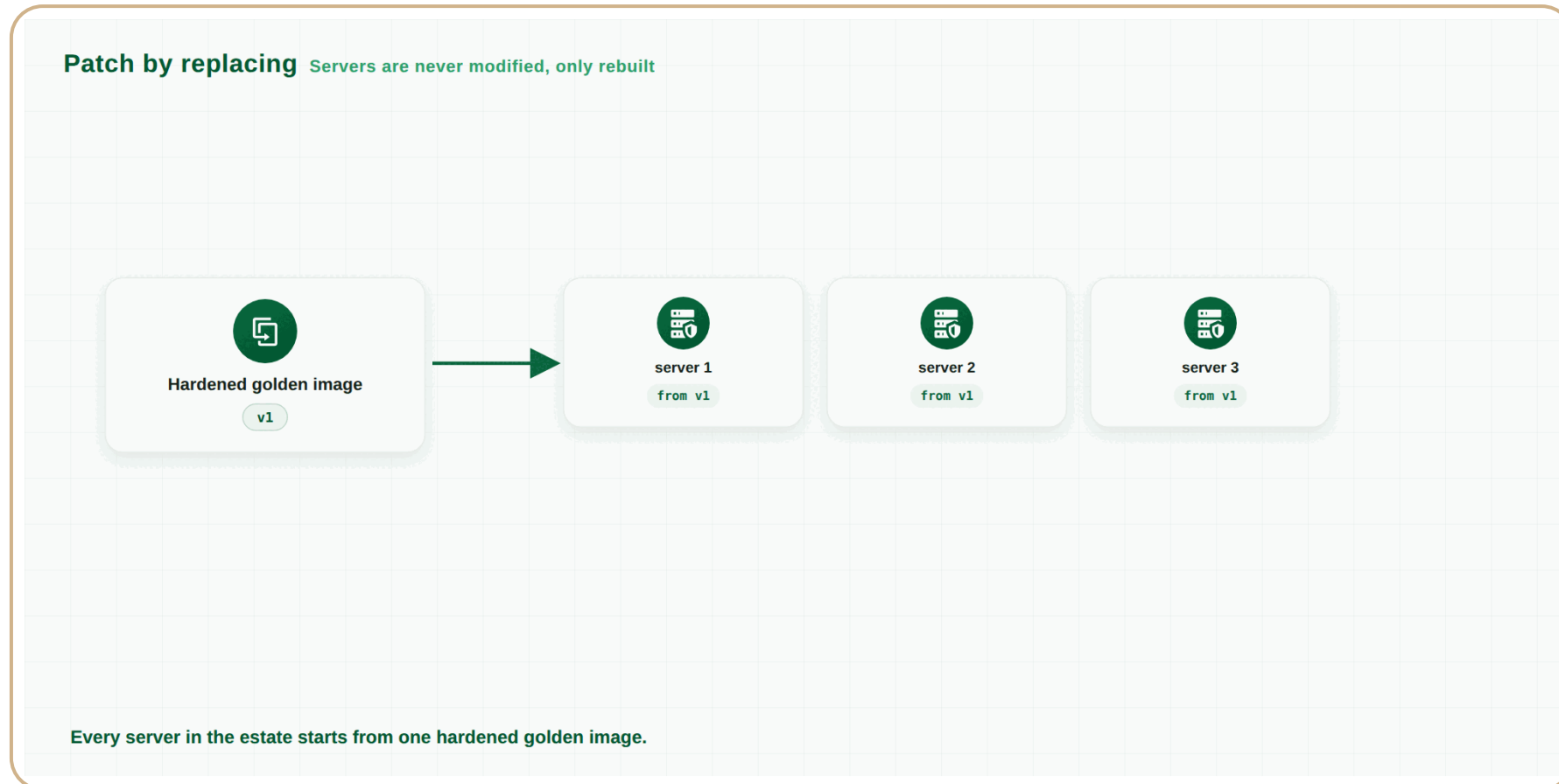
Secure defaults

Encryption on, public access off, strong TLS, from the very first minute. Safe unless deliberately opened.

CIS Benchmarks (Center for Internet Security). Golden-image and immutable-infrastructure practice.



Never patch a server, replace it



Patching one by one always leaves the server that was missed. Rebuilding the golden image and replacing every instance leaves nothing to drift.

Which foundation fixes the sin?

Six common cloud sins on the left. Match each to the foundational control that prevents it on the right.

The sin

1. A storage bucket is public to the whole internet
2. The root account has an access key and no MFA
3. A VM was launched from an old, unpatched image
4. Nobody noticed a firewall was opened last week
5. A team spun up costly resources in a banned region
6. The same misconfiguration returns after each fix

The foundation

- A. Posture management (secure score)
- B. Identity hardening (MFA, no root keys)
- C. Hardened baseline / golden image
- D. Logging and monitoring
- E. Org guardrail (SCP / Azure Policy)
- F. Policy-as-code auto-remediation



PART B · FOUNDATIONS 3 AND 4

See everything: visibility and posture

You cannot secure, or even find, what you cannot see.



Visibility: turn on all the logs

In an ephemeral, API-driven environment, logs are your only memory. Enable them everywhere, from day one, before you need them.

What you need to see	AWS	Azure
Who did what (audit)	AWS CloudTrail	Azure Activity Log
Is the config compliant	AWS Config	Azure Policy / Resource Graph
Metrics and app logs	Amazon CloudWatch	Azure Monitor
One place to see it all	AWS Security Hub	Microsoft Defender for Cloud

Answering warm-up S2: you cannot investigate an incident with logs you only switched on afterwards.



Posture management: catch misconfiguration

Cloud Security Posture Management (CSPM) continuously checks your live configuration against benchmarks and flags what drifts. It is the direct answer to the number one cloud threat.



Continuous assessment

Every resource is checked against CIS and vendor benchmarks, around the clock, with no manual audit.



A single secure score

Your whole estate distilled to one number and a ranked list of what to fix to raise it.



Prioritised findings

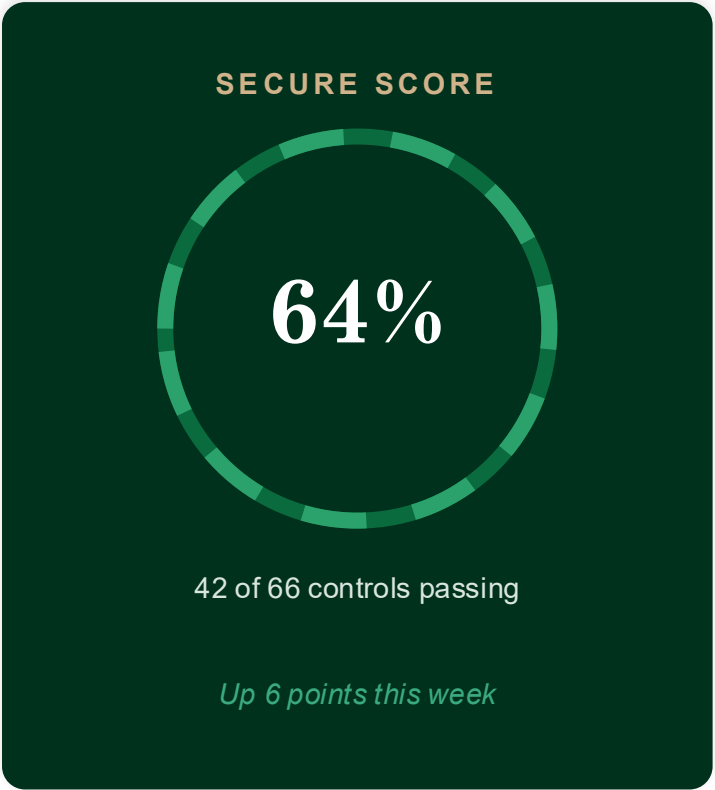
Each misconfiguration comes with a severity and step-by-step remediation, so you fix the worst first.

The tools: AWS Security Hub and Microsoft Defender for Cloud are CSPM platforms. Both give a secure score, both check against CIS, and both tell you exactly what to change. This is how you stop misconfiguration becoming a breach.

AWS Security Hub; Microsoft Defender for Cloud (formerly Azure Security Center).



Reading a secure-score dashboard



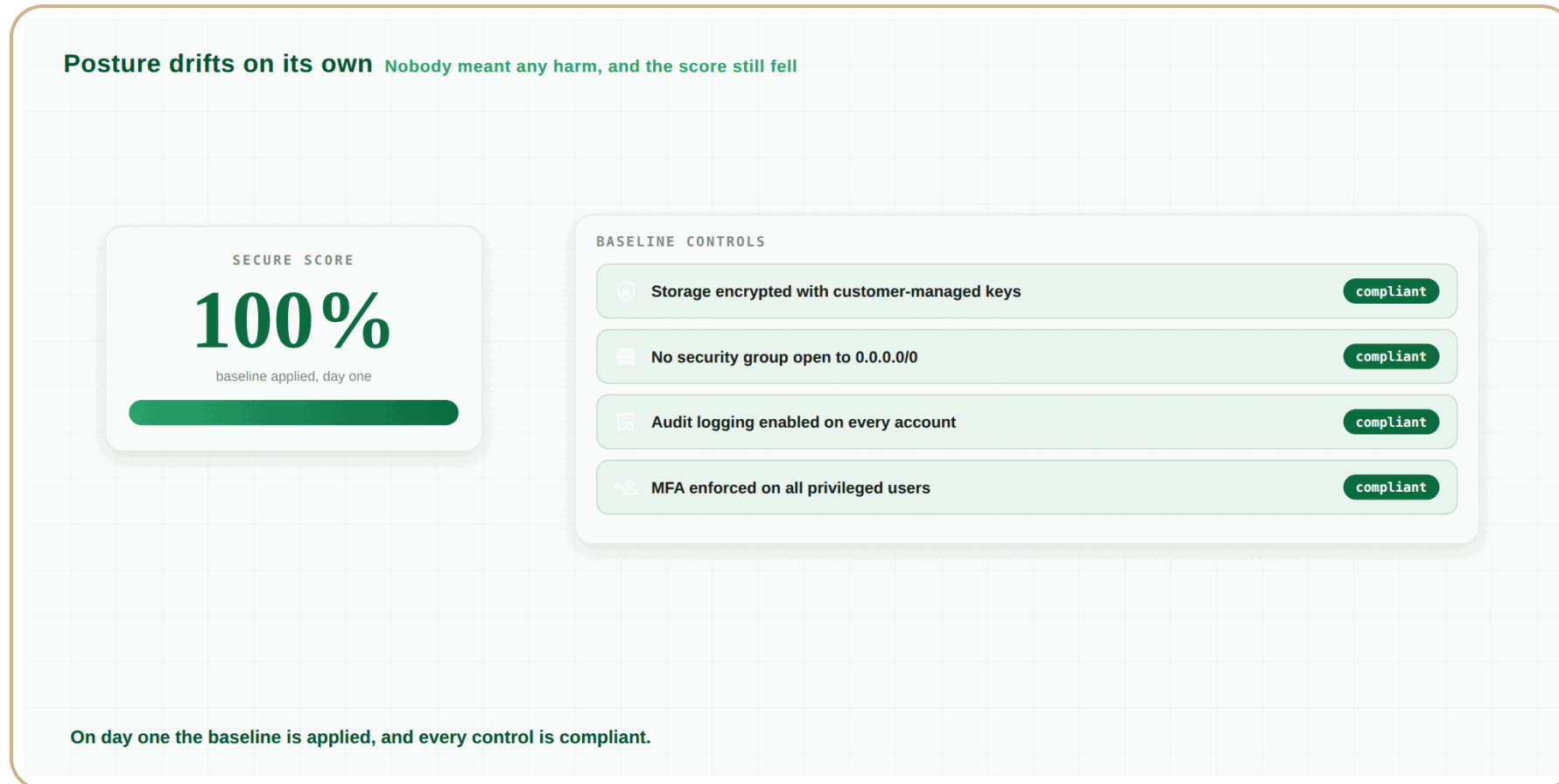
Top findings, worst first

CRITICAL	Root account has an active access key	<i>Delete the key; use roles instead</i>
HIGH	Storage bucket is publicly readable	<i>Block public access; restrict policy</i>
HIGH	Audit logging disabled in 2 regions	<i>Enable CloudTrail / Activity Log everywhere</i>
MEDIUM	14 virtual machines missing OS patches	<i>Rebuild from the patched golden image</i>
MEDIUM	Password policy allows 6 characters	<i>Require 14+ and MFA</i>

Illustrative dashboard in the style of AWS Security Hub and Microsoft Defender for Cloud. You will use the real thing in the labs.



The score falls on its own



Nobody meant any harm: a port opened, a log switched off, and the estate drifted. Continuous posture scanning catches both, and remediation puts them back.

Triage the findings

You have limited time and cannot fix everything today. Rank these five findings, worst first, and write one line on why your number one is the most urgent.

CRITICAL

Root account has an active access key

HIGH

A storage bucket with customer data is public

HIGH

Audit logging is off in two regions

MEDIUM

14 virtual machines are missing OS patches

MEDIUM

The password policy allows 6-character passwords

There is a defensible order. We settle it on the verdicts slide.



PART C · FOUNDATIONS 5 AND 6

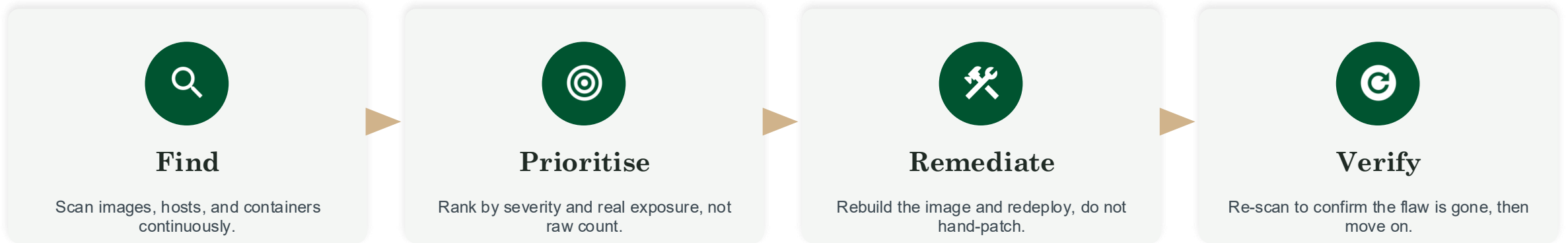
Find and fix: vulnerabilities and automation

Then make the fix stick, so the same hole never opens twice.



Vulnerability management and remediation

Posture catches misconfiguration; vulnerability management catches known software flaws (CVEs) in your images, packages, and containers. Same loop: find, prioritise, fix.

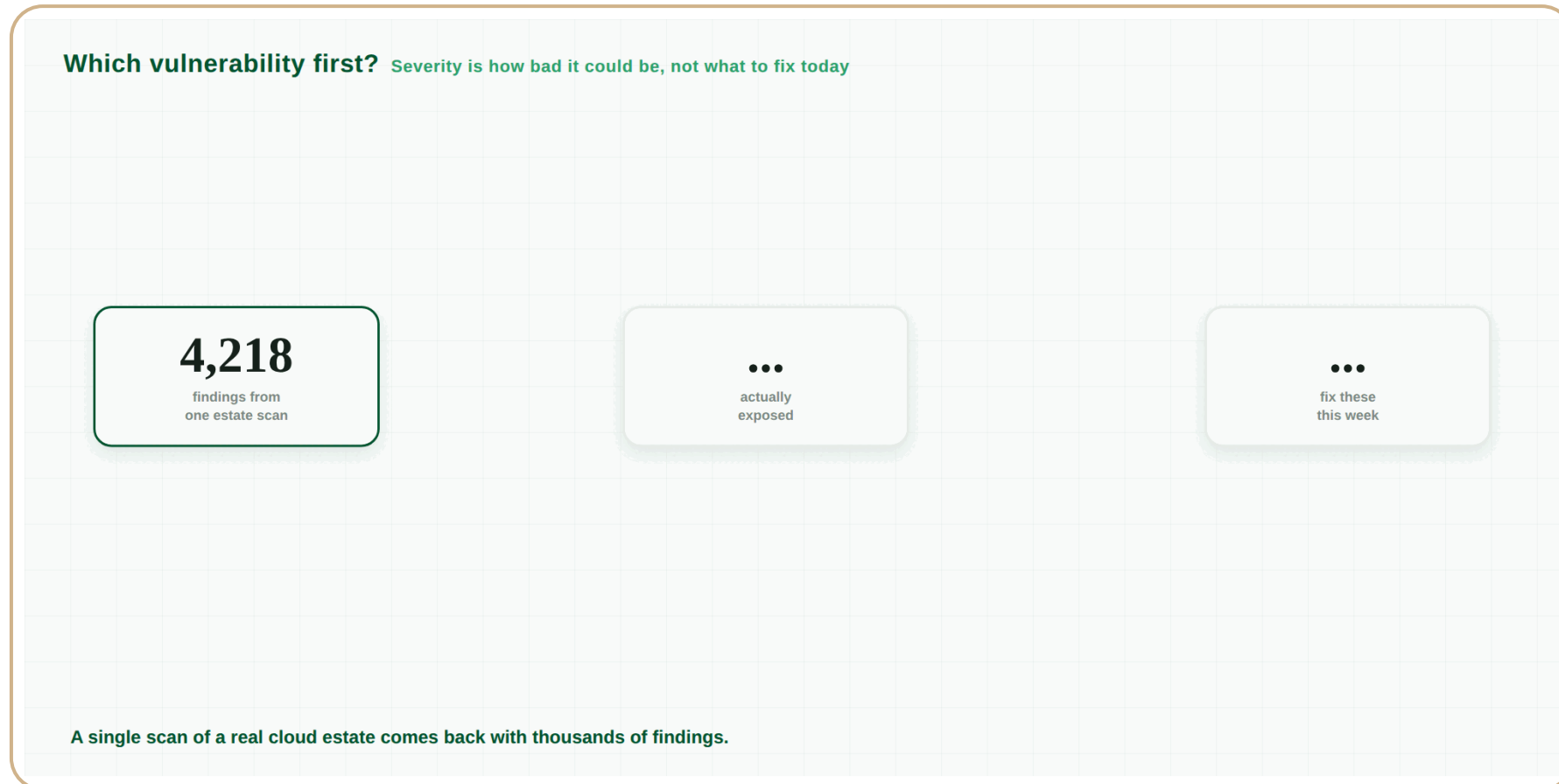


The tools, and the cloud twist: Amazon Inspector and Microsoft Defender for Cloud scan for vulnerabilities automatically. The cloud advantage is immutable infrastructure: instead of risky live patching, you rebuild the golden image clean and replace the servers. This is why warm-up S3 is a misconception.

Amazon Inspector; Microsoft Defender for Cloud vulnerability assessment.



Four thousand findings, twenty-three that matter



Severity alone still leaves hundreds of criticals. Filtering by what is actually reachable, then by what is being exploited, turns the backlog into a week of work.

Automation: fix once, enforce forever

Manual security does not scale to thousands of ephemeral resources. The foundation that holds all the others together is automation: security expressed as code.



Auto-remediation

A rule detects a public bucket and closes it in seconds, with no human in the loop.



Policy as code

Guardrails written as code (AWS Config rules, Azure Policy) enforce themselves across every account.



Scan before deploy

Check infrastructure-as-code templates for insecure settings before they ever reach production.



Prevent recurrence

When a fix is encoded as policy, the same mistake cannot come back. That is how posture improves for good.

Answering warm-up S1: security is not a one-time setup. Automation is what keeps the foundation secure as it changes.



Policy as code, catching it before deployment

```
# Policy as code: catching the misconfiguration before it is ever deployed

devops@pipeline:~/infra$ grep -nE 'acl|cidr_blocks' bad/main.tf
3: acl      = "public-read"
13: cidr_blocks = ["0.0.0.0/0"]

devops@pipeline:~/infra$ checkov -d bad --compact --quiet --skip-download --check $CHK 2>/dev/null | grep -vE '^\\s+File:|^$'
terraform scan results:
Passed checks: 0, Failed checks: 5, Skipped checks: 0
Check: CKV_AWS_24: "Ensure no security groups allow ingress from 0.0.0.0:0 to port 22"
  FAILED for resource: aws_security_group.admin
Check: CKV2_AWS_6: "Ensure that S3 bucket has a Public Access block"
  FAILED for resource: aws_s3_bucket.records
Check: CKV_AWS_20: "S3 Bucket has an ACL defined which allows public READ access."
  FAILED for resource: aws_s3_bucket.records
Check: CKV_AWS_21: "Ensure all data stored in the S3 bucket have versioning enabled"
  FAILED for resource: aws_s3_bucket.records
Check: CKV_AWS_145: "Ensure that S3 buckets are encrypted with KMS by default"
  FAILED for resource: aws_s3_bucket.records

# The same policy, run again after the Terraform is fixed
devops@pipeline:~/infra$ checkov -d good --compact --quiet --skip-download --check $CHK 2>/dev/null | grep -vE '^\\s+File:|^$'
terraform scan results:
Passed checks: 5, Failed checks: 0, Skipped checks: 0

# The bucket was never public, because the pipeline refused to deploy it.
```

A genuine scan: the public-read bucket and the SSH rule open to the internet fail five policies, and the same policies pass once the Terraform is fixed.



Security posture is a cycle, not a project

A secure foundation is never finished. These five steps run continuously, because the cloud, and the threats, change every day.



◀ *Enforce feeds straight back into Baseline. The loop never stops.*

Drift is normal: someone opens a port, a new service launches, a benchmark updates. The loop catches it before an attacker does.



A secure foundation, in one checklist

Day one of any cloud estate. Each item maps to a foundation and exists in both AWS and Azure.

- ✓ Lock the root / global-admin account and enforce MFA everywhere
- ✓ Set org-wide guardrails (SCPs / Azure Policy) and a landing zone
- ✓ Launch only from hardened, CIS-benchmarked golden images
- ✓ Turn on every audit and config log, in every region
- ✓ Enable CSPM (Security Hub / Defender for Cloud) and watch the secure score
- ✓ Scan continuously for vulnerabilities, and remediate by redeploying
- ✓ Express guardrails and fixes as code, with auto-remediation

Get these seven right and you have closed the doors behind most cloud breaches.



Your four votes, settled

S1

Once set up securely, it stays secure

Misconception. Configuration drifts constantly. Posture is a continuous loop, held in place by automation.

S2

Enable logging only if there is an incident

Misconception. You cannot investigate with logs you never collected. Turn everything on first, from day one.

S3

Patch cloud servers like a laptop, one by one

Misconception. You rebuild the hardened image and redeploy. Immutable infrastructure beats hand-patching.

S4

100 percent secure score means unhackable

Misconception. Secure score is configuration hygiene, not threat detection. Necessary, never sufficient.



Chapter 2 recap



The mindset Secure by default, defense in depth, least privilege, assume breach. The frameworks behind every choice.



Prevent by default Guardrails and hardened baselines stop whole classes of mistake before anyone can make them.



See and assess Turn on all logging, then let CSPM continuously score your configuration against benchmarks.



Find, fix, enforce Manage vulnerabilities and automate remediation, so the same hole never reopens. Posture is a loop.



CHAPTER 2 · THE FOUNDATION

A secure cloud is built once and maintained forever.

Next: Chapter 3, identity and access management, the strongest lock on the door and the one attackers pick first.

